

Day 2: Incident Reasoning Brief

CareerForge: The System Behind The Bug

Variant ID: A

Name: ALIA ISLAM

College: VIDYALANKAR POLYTECHNIC Branch / Year: IF/ 3RD YEAR

1. Founder Assumption

"Recruiters are saying shortlist actions are delayed and candidates are not getting notifications. This looks like a notification or recruiter action issue. Fix that first."

What did the founder assume?

The founder assumed that delayed candidate notifications and slow recruiter shortlist actions were the root cause, not downstream effects of a deeper failure. The visible symptoms: *a noisy notification queue and stalled UI actions*, were treated as the origin of the incident.

Was the assumption correct, partially correct, or misleading?

Misleading.

The founder's assumption was misleading because it focused on visible surface-level symptoms rather than the underlying origin failure. Notifications were delayed and that part is true, but notifications were delayed because the PostgreSQL database was struggling to commit writes, not because the notification system itself was broken.

Fixing notifications without addressing the database write latency would have been cosmetic: the queue would remain backed up, recruiter actions would still retry and duplicate, and candidate statuses would remain inconsistent. The source of truth was already degraded before any notification was sent.

2. Competing Hypotheses

Hypothesis	Evidence Supporting It	Evidence Against It	Accept / Reject / Modify
H1: Notification queue is the root cause	Notification delay is visible and noisy (70–150s vs normal 5–20s). Candidates reported late or missing status updates. Founder flagged it first.	Logs show DB write latency spiked at 10:01:10 BEFORE the notification delay warning at 10:01:22. Notification queue can only dispatch after status is committed, it is downstream of the DB.	REJECT: Notification delay is a symptom of DB latency, not the origin.
H2: Recruiters are double-clicking (human error)	39 duplicate shortlist reports/hour (vs 1–2 normal).	Duplicate actions began only after DB write latency spiked. The retry	REJECT as root cause Recruiter retries are a rational human response

	Recruiter complaint: 'I clicked again because I thought the action did not save.'	logic triggered because the UI received no timely confirmation, recruiters responded rationally to a slow system. Log confirms: retry started at 10:01:14, after DB latency warning at 10:01:10.	to a slow system, not the origin.
H3: PostgreSQL DB write latency is the root cause	DB write latency jumped from 80–250ms to 3,500–7,800ms. Log: [10:01:10] WARN DB write latency high. All downstream failures (retries, duplicates, notification delay, dashboard lag, analytics mismatch) appear in the logs AFTER this line. Review traffic spiked 5.7x (90 → 510 actions/min) — consistent with a DB connection pool saturation.	No direct log of connection pool exhaustion (could confirm with DB-level monitoring). Latency alone doesn't confirm the exact DB failure mode.	ACCEPT: DB write latency is the most consistent origin failure aligned with all evidence.

3. Origin Failure

What most likely failed first:

PostgreSQL candidate_status write latency collapse, triggered by a 5.7x traffic spike from the hiring deadline review sprint.

Confidence: High

Why this confidence level:

The system log provides a clear temporal chain. The DB write latency warning at 10:01:10 precedes every downstream symptom. No other failure appears before this timestamp. The traffic volume (510 actions/min vs. 90 normal) is a plausible mechanical cause for DB saturation. All three competing hypotheses for alternative root causes are contradicted or fully explained by the DB latency.

Evidence:

- E1: [10:01:10] WARN DB write latency high table=candidate_status duration_ms=5100, DB latency is the first WARN/ERROR in the log sequence, predating all other downstream warnings.
- E2: Review traffic spike from 90 to 510 actions/minute (5.7x increase) detected at 10:01:02 establishes the mechanical pressure that likely exhausted the DB connection pool or write throughput.
- E3: Recruiter action retry started at 10:01:14, duplicate shortlist suspected at 10:01:16, notification queue delay at 10:01:22, Redis mismatch at 10:01:20, all appear after the DB warning, consistent with cascading downstream failure from a single upstream bottleneck.

4. Symptom vs Root Cause Table

Reported Issue	Root Cause / Downstream Effect	Evidence
Recruiter actions not updating	Downstream effect: DB write latency means status changes are not committed in time for the UI to confirm the action.	[10:01:10] WARN DB write latency high duration_ms=5100; [10:01:13] WARN Candidate status record not committed yet
Duplicate shortlist actions	Downstream effect: Recruiter UI received no acknowledgment within timeout, triggering retry logic. The system processed the same intent twice.	[10:01:14] WARN Recruiter action retry started; [10:01:16] ERROR Duplicate shortlist action suspected; Duplicate reports: 39/hour vs 1–2 normal
Notification delays	Downstream effect: Notification Queue depends on a committed DB status before dispatching. Slow writes = delayed trigger.	[10:01:13] WARN status not committed; [10:01:22] WARN Notification queue delay: 132 seconds; Queue delay 70–150s vs 5–20s normal
Candidate status mismatch	Downstream effect: Redis cache served a status that the DB had not yet committed, creating a split view between cache and source-of-truth.	[10:01:20] WARN Redis status mismatch cache_status=shortlisted db_status=pending_commit; 16% analytics mismatch window
Dashboard lag	Downstream effect: Dashboard reads from Redis and PostgreSQL fallback. Both are either stale (Redis) or slow to respond (DB under load).	Dashboard lag: 1–4 min vs under 5s normal; [10:01:23] WARN Recruiter dashboard sync delayed

Analytics inconsistency	Downstream effect: Analytics records the click event immediately but cannot match a committed status event because the DB write has not completed.	[10:01:29] shortlist_clicked event received; [10:01:30] WARN missing candidate_status_committed event; 16% mismatch in current window
-------------------------	--	---

5. Failure Propagation Chain

Origin failure:

PostgreSQL write latency collapses under 5.7x traffic spike (90 → 510 actions/min)

[10:01:10] DB write latency 5,100ms (normal: 80–250ms)

->

Affected component: Candidate Review Service

Status writes queue up; commit confirmations delayed or missing

[10:01:13] Candidate status record not committed yet

->

Affected component: Recruiter UI / Backend API

No action acknowledgment received within timeout → recruiter retries

[10:01:14] Recruiter action retry started

[10:01:16] Duplicate shortlist action suspected

->

Affected component: Redis Status Cache

Cache reflects optimistic/stale state; DB has not committed matching record

[10:01:20] Redis mismatch: cache=shortlisted, db=pending_commit

->

Affected component: Notification Queue

Cannot dispatch notification until status is committed; queue builds up

[10:01:22] Notification queue delay: 132 seconds

->

Affected component: Recruiter Dashboard + Analytics

Dashboard counts disagree with queue; analytics missing committed events

Dashboard lag: 1–4 min; 16% analytics mismatch

->

User-facing symptom:

Candidates receive late/contradictory status emails

Recruiters see frozen UI, double-count shortlists

Hiring lead sees wrong dashboard numbers

->

Business impact:

Risk of duplicate shortlists violating the non-negotiable hiring constraint

Risk of incorrect shortlist count reaching the 6:00 PM hiring deadline

Loss of recruiter trust in the console

6. First 15-Minute Incident Actions

Action	Why Now	Risk Reduced	Tradeoff
Throttle or rate-limit recruiter review actions to reduce DB write pressure	The DB is under 5.7x normal load. Reducing inflow is the fastest way to let the write queue drain and latency recover without a deployment.	Stops the duplicate action spiral. Prevents further uncommitted writes from backing up. Gives DB headroom to clear existing queue.	Recruiter throughput slows temporarily. Some recruiters will be frustrated by the wait.
Pause or disable recruiter retry auto-trigger in the UI (show 'Action pending' instead of silently retrying)	Retries are amplifying write volume on an already saturated DB. Each retry adds another write attempt, worsening latency.	Eliminates duplicate shortlist creation. Reduces DB write volume. Gives the DB a chance to recover.	Recruiter UX is degraded, they see a 'pending' state rather than immediate confirmation. Acceptable versus data corruption.
Pause non-critical notification dispatch for candidates whose status is still in pending_commit state	Sending notifications before status is committed causes contradictory messages to candidates (shortlisted email + portal says Under Review). This violates the non-negotiable of not showing candidates a contradictory status.	Prevents misleading candidate communications. Reduces downstream support volume.	Candidate notifications are delayed further. Acceptable, delay is better than sending wrong information.

7. What I Would Not Fix Immediately

Delayed Work	Why Delay	Risk Accepted
Dashboard freshness / live count accuracy	Dashboard lag is a visibility problem, not a data integrity problem. Stale counts do not corrupt the source of truth — they just frustrate the hiring lead. DB stability must come first.	Hiring lead sees incorrect counts for longer. Manageable if they are informed of the ongoing incident.
Analytics event reconciliation (16% mismatch)	Analytics are a reporting concern, not a live correctness concern. Missing committed events in the analytics window will persist but do not affect recruiter actions or candidate outcomes directly.	Hiring deadline report may show undercounts. Acceptable, can be corrected post-incident with a backfill from the DB.
Notification queue backlog clearance	Once the DB is stable and statuses are committed, the notification queue will	Candidates experience continued notification delays.

	self-drain. Manually intervening in the queue before DB recovery risks sending notifications against still-uncommitted statuses.	Acceptable, late notifications are better than wrong notifications.
--	--	---

8. Tradeoff Ownership

What tradeoff are you accepting?

By throttling recruiter throughput and pausing retries, I am accepting that recruiters will be slowed down during an already deadline-pressured sprint. The system will visibly feel slower and less responsive for 15–30 minutes. This is a deliberate choice to protect data integrity over perceived performance.

Who is helped first?

- The hiring data integrity: no new duplicates, no further uncommitted status writes poisoning the cache or analytics.
- Candidates: they stop receiving contradictory status notifications.
- The hiring lead: the final shortlist count becomes more trustworthy once the DB drains and commits correctly.

Who still experiences pain temporarily?

- Recruiters: their workflow is slowed. Actions will show 'pending' rather than confirming instantly.
- The founder: the dashboard will still lag, which is visible and may draw complaints.
- Candidate support: the notification backlog will continue to drain slowly before clearing.

This is the correct tradeoff. A 15-minute slowdown for recruiters is recoverable. A corrupted shortlist at 6:00 PM is not.

9. AI / Assumption Challenge

Initial suggestion (founder / first instinct)

The founder suggested fixing notifications first. First instinct, seeing the noisy duplicate shortlist reports and dashboard mismatch, might have been to look at the UI caching layer or the notification queue as the broken component.

Evidence confirmed / rejected it

Both were rejected as root causes by the system log timestamp sequence. The DB write latency warning at 10:01:10 is the earliest anomalous event. Every other symptom, retry at 10:01:14, duplicate at 10:01:16, Redis mismatch at 10:01:20, notification delay at 10:01:22, appears after it. No evidence supports any failure originating outside the DB layer before that timestamp.

What changed my mind

The chronological ordering of the log entries. A system where notifications failed first would show a notification queue error before a DB write warning. That is not what the logs show. The DB warning is first, everything else follows. The 5.7x traffic spike provides the mechanical explanation for why the DB failed at that moment.

10. Final Incident Judgment:

The PostgreSQL database collapsed under a 5.7x traffic spike from the hiring deadline review sprint, causing write latency to jump from under 250ms to over 7,800ms. This single failure cascaded: recruiter actions timed out and retried (creating duplicates), Redis served stale statuses, the notification queue backed up waiting for committed writes, and the dashboard showed incorrect counts. The founder assumed notifications were the root cause, they are not; they are the most visible symptom. The correct stabilization priority is to reduce DB write pressure first (throttle recruiter actions, disable auto-retries), which will allow the write queue to drain, Redis to re-sync, and notifications to dispatch correctly. Recruiters wait 15 minutes; the alternative is a corrupted shortlist at the 6:00 PM deadline.